

Classification Models (Theory)

Welcome to Classification Models! In this session, we transition from predicting continuous numerical values (Regression) to predicting discrete categories or labels (**Classification**). You will learn the mechanics of **Logistic Regression** and **Decision Trees**, understand the structure of a **Confusion Matrix**, and explore core evaluation metrics along with guidance on when to prioritize each in real-world scenarios.

1. Mechanics of Classification Models

Unlike regression, which outputs an unbounded number, classification models map input features to a discrete class label (e.g., `Spam` vs. `Not Spam`, or `Churn` vs. `Retained`).

A. Logistic Regression

Despite its name containing "regression," **Logistic Regression** is a fundamental linear classification algorithm used for binary outcomes (0 or 1, Yes or No).

- **The Sigmoid Function:** Instead of fitting a straight line directly to the data, Logistic Regression passes the linear output through the **Sigmoid function** (*S*-curve), compressing any real-valued number into a probability between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where $z = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$.

- **Decision Threshold:** A threshold (typically 0.5) is applied. If the predicted probability is ≥ 0.5 , the model classifies the instance as Class 1; otherwise, Class 0.

B. Decision Trees

A **Decision Tree** is a non-linear, tree-structured model made up of nodes, branches, and leaves.

- **Recursive Splitting:** The algorithm evaluates all features to find the split that best separates the data into distinct classes, measured using metrics like **Gini Impurity** or **Information Gain (Entropy)**.

- **Structure:**
 - **Root Node:** The starting feature split.
 - **Internal Nodes:** Subsequent feature conditions (e.g., `Is Monthly Charges > $50?`).
 - **Leaf Nodes:** The final predicted class category.
-

2. The Confusion Matrix

A **Confusion Matrix** is a tabular summary of prediction results for a classification problem, comparing model predictions against actual ground-truth labels. For a binary classifier, it consists of four quadrants:

Metric	Description
True Positive (TP)	Model predicted Positive , and it was actually Positive .
True Negative (TN)	Model predicted Negative , and it was actually Negative .
False Positive (FP)	Model predicted Positive , but it was actually Negative (Type I Error).
False Negative (FN)	Model predicted Negative , but it was actually Positive (Type II Error).

3. Evaluation Metrics

Derived directly from the confusion matrix, these metrics measure different aspects of classifier performance:

A. Accuracy

The ratio of correct predictions to total predictions made.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- *Limitation:* Can be misleading if dataset classes are heavily imbalanced (e.g., 99% non-fraudulent transactions).

B. Precision

Out of all instances predicted as Positive, how many were actually Positive? Focuses on minimizing False Positives.

$$\text{Precision} = \frac{TP}{TP + FP}$$

C. Recall (Sensitivity)

Out of all actual Positive instances, how many did the model manage to find? Focuses on minimizing False Negatives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

D. F1-Score

The harmonic mean of Precision and Recall, providing a balanced single score when balancing false positives and false negatives is critical.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

4. Scenario-Based Metric Selection

Choosing the right evaluation metric depends entirely on the business cost of making mistakes (False Positives vs. False Negatives):

Scenario 1: Email Spam Detection

- *Risk:* A legitimate email is marked as spam (False Positive).
- *Priority:* **Precision**. You want to be 100% sure an email is spam before hiding it from the user's main inbox.

Scenario 2: Medical Disease Screening (e.g., Cancer Detection)

- *Risk:* A sick patient is incorrectly diagnosed as healthy (False Negative), delaying critical treatment.
- *Priority:* **Recall**. It is better to flag a few false alarms (false positives) than miss a single positive case.

Scenario 3: Balanced Customer Churn Prediction

- *Goal:* Balance customer retention marketing costs against lost subscriptions.
 - *Priority:* **F1-Score**. Ensures a healthy trade-off between precision and recall when classes are moderately balanced.
-

5. Reference Links & Further Reading

- [Scikit-Learn Logistic Regression Guide](#): Official documentation and configuration parameters.

- **Scikit-Learn Classification Metrics API:** Guide on calculating accuracy, precision, recall, and confusion matrices in Python.

confusion matrix

A confusion matrix is an $N \times N$ table used to evaluate the performance of a classification model by comparing its predicted labels against the actual true labels. It breaks down the total number of correct and incorrect predictions into distinct categories, allowing you to see exactly *how* a model is failing, not just how often.

1. The Confusion Matrix & Sample Data

For a standard binary classification problem (predicting exactly two possible outcomes), the matrix is a 2×2 grid.

Let's use a **Spam Email Filter** as our sample scenario:

- **Positive Class (1):** The email is Spam.
- **Negative Class (0):** The email is Not Spam (Normal).

Suppose we test our model on a dataset of 1,000 emails:

	Predicted: Spam (Positive)	Predicted: Not Spam (Negative)
Actual: Spam (Positive)	True Positive (TP): 90 (Spam correctly caught)	False Negative (FN): 10 (Spam missed, goes to inbox)
Actual: Not Spam (Negative)	False Positive (FP): 5 (Normal email sent to spam)	True Negative (TN): 895 (Normal email stays in inbox)

2. Evaluation Metrics

Using the four values from the matrix (TP, TN, FP, FN), we can calculate several crucial performance metrics.

Accuracy The ratio of correctly predicted observations to the total observations.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{90 + 895}{1000} = 98.5\%$$

Precision (Positive Predictive Value) Out of all the emails the model *predicted* as spam, how many were *actually* spam? Precision penalizes False Positives.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{90}{90 + 5} = 94.7\%$$

Recall (Sensitivity or True Positive Rate) Out of all the *actual* spam emails in the dataset, how many did the model *successfully find*? Recall penalizes False Negatives.

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{90}{90 + 10} = 90.0\%$$

F1-Score The harmonic mean of Precision and Recall. It provides a single score that balances both concerns, especially useful when class distribution is uneven.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \times \frac{0.947 \times 0.900}{0.947 + 0.900} = 92.3\%$$

Show [Confusion Matrix & Metrics](#)

3. Scenario-Based Metric Selection

Accuracy is a terrible metric if your data is highly imbalanced (e.g., if 99% of transactions are legitimate, a model that just guesses "legitimate" every time is 99% accurate but completely useless for finding fraud).

You must choose your metric based on the **cost of making a specific type of error**.

- **Optimize for Precision when False Positives are highly costly.**
 - *Example: Spam filters.* It is far worse to send a critical work email (False Positive) to the spam folder than it is to accidentally let a junk email (False Negative) into the inbox.
 - *Example: Criminal justice predictions.* Falsely accusing an innocent person (FP) is generally considered worse than failing to flag a guilty one (FN).
- **Optimize for Recall when False Negatives are highly costly.**
 - *Example: Cancer screening.* It is far worse to tell a sick patient they are healthy (False Negative - missed diagnosis) than to tell a healthy patient they might be sick and need a follow-up test (False Positive).
 - *Example: Fraud detection.* Missing a stolen credit card charge (FN) costs the bank money; flagging a normal purchase (FP) is just a minor annoyance for the customer.
- **Optimize for F1-Score when you need a balance or have imbalanced classes.**
 - *Example: Search Engine Results.* You want the results to be highly relevant (Precision) but you also want to make sure you return most of the relevant documents that exist (Recall).

Key insight: You almost always have to trade Precision for Recall. If you make a model ultra-sensitive so it catches *every single* case of cancer (100% Recall), it will inevitably flag many healthy people along the way (lowering Precision).
